MECH321
Dynamics of Engineering Systems

Lecture 10 · Optional
PID Control

Prof. Zengxi (Stephen) Pan

TYUT · 2026

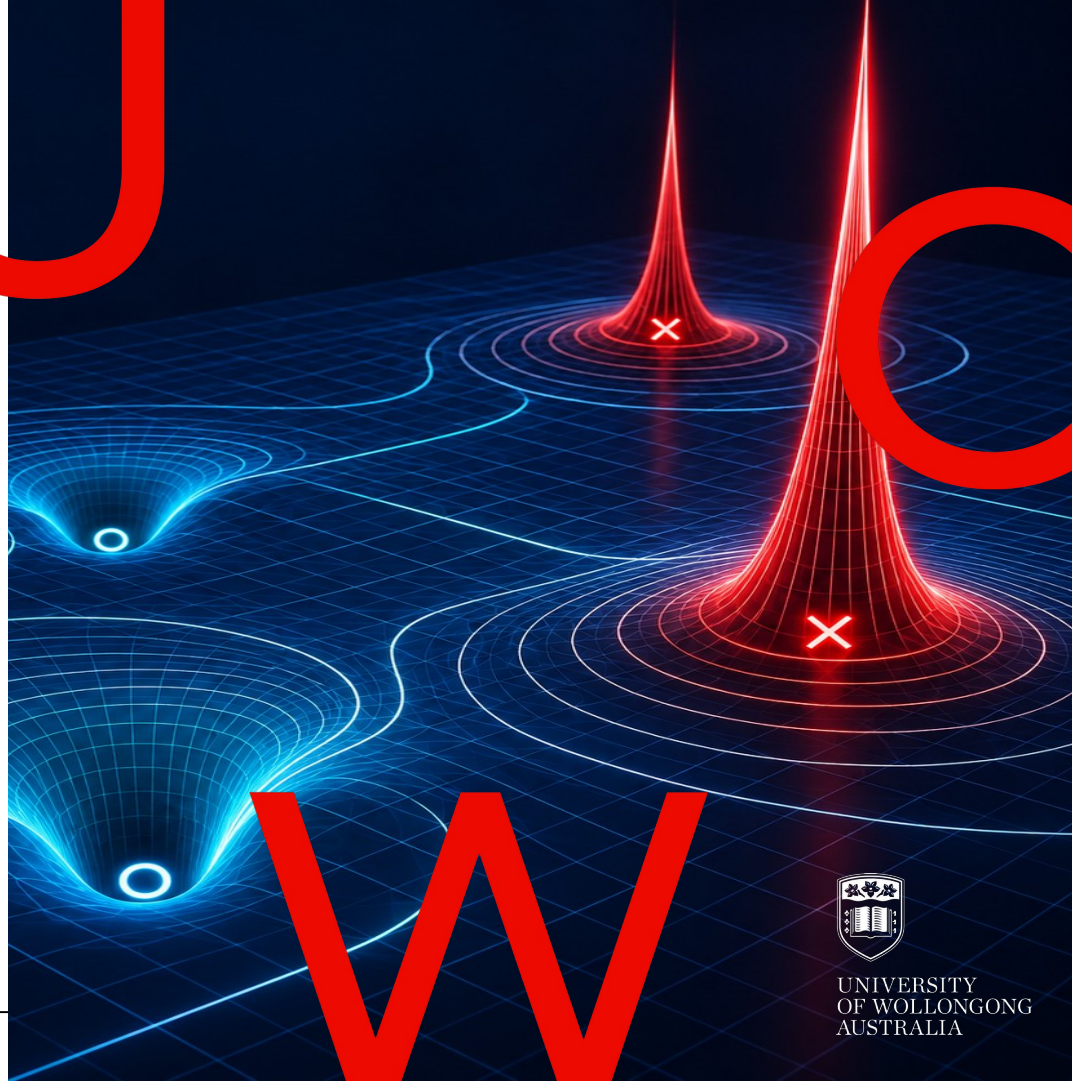


Table of Contents

PART 1 **PID actions and controller choice**

PART 2 **Controller design and implementation**

PART 3 **Practice and implementation extensions**

After Lecture 9's s stability checks, compare P, PI and PID under actuator and sensor limits to make a feasible final design choice.

PART 1

PID actions and controller choice

Compare controller actions and calculate a complete proportional response.

01 P I and D actions

02 Controller comparison

03 Concept check

Preparation check

OPTIONAL

Home Reading 10

A Controller actions

What does each of the proportional, integral and derivative actions respond to, and how does each change the response?

B Integral trade-off

Why can integral action remove step error yet change the closed-loop poles and reduce stability margin?

C Physical limits

How can actuator saturation and integrator windup defeat an otherwise stable linear controller design?

Answer independently, then compare explanations. Record any term or step you cannot yet explain.

The parallel PID law

OPTIONAL

Controller gains—not Lecture 7 tracking-error constants

$$u(t) = K_p e(t) + K_i \int e(t)\,dt + K_d \frac{de}{d}$$

$$C(s) = K_p + \frac{K_i}{s} + K_d s$$

Proportional
the present
Reacts to the error that exists now
Bigger error, bigger correction

Integral
the past
Accumulates error over time
Keeps pushing until the error is gone

Derivative
the future
Reacts to how fast the error is changing
Brakes before the target is reached

State the form before you quote a gain

Many texts write $K_p[1 + \frac{1}{T_i s} + T_d s]$ instead. The same controller has completely different numbers in the two conventions, so a gain without its form is not information.

Proportional action

OPTIONAL

Effects of P gain depend on the plant and the chosen output

What raising K_p does

For stable Type 0 feedback, it reduces finite step error

For $G(s) = 2 \quad /(s+2)$, the pole is $-(2+2K_p)$

Positive K_p speeds this first-order response without overshoot

Extra poles or delay can instead limit the stable gain

Why the error never reaches zero

With $C = K_p$, the error is $\frac{1}{1+K_p G(0)}$

Zero error would need infinite gain

And a finite error is required to produce any output at all — that is the mechanism, not a defect

Lecture 8's damping trade-off is model-specific

For its position plant $1/[s(s+2)]$, more positive gain reduces damping and raises overshoot, but the ideal closed loop stays stable. An eventual instability needs a different plant, extra dynamics or delay.

P control: calculate the entire step

EXAMPLE

A single pole still leaves a finite offset for this type-0 plant

Problem and closed loop

Take $G(s) = 2$ /(s+2), unity feedback and $C(s)$=K. From 1+KG, the closed-loop transfer is 2K/(s+2+2K). At $K = 4$ its pole is −10 rad/s, so the time constant is 0.10 s. The DC gain is 8/10=0.8.

Time response and physical meaning

For a unit step, $y(t) = 0.8$ (1−e^(−10t)). Thus $y(0) = 0$, y(∞)=0.8 and the final error is 0.2. The controller needs a nonzero error to provide the steady command: u(∞)=4×0.2=0.8. Raising K reduces the offset but also raises the initial request $u(0^+)$=K.

CHECK **The final-value calculation and the explicit time response must agree.**

Integral action

OPTIONAL

What it buys

An uncancelled integrator raises the loop type by one

Zero step error requires a stable loop and a feasible command

Integral output can remain nonzero when error is zero

Constant-load rejection depends on its entry path

What it costs

That pole at the origin adds 90° of phase lag

Which comes straight out of your phase margin

Aggressive K_i gives slow oscillation, then instability

And it introduces windup — see slide 16

The trade, using Lecture 8 stability margins

Check phase margin at the new crossover. Integral action can remove steady error only when the loop remains stable and the actuator can supply the required output.

Derivative action

OPTIONAL

What it buys

Adds phase lead, up to +90°

Which is exactly what integral action took away

Improves damping, so overshoot falls

Allows a higher crossover frequency, hence a faster loop

What it costs

An ideal differentiator has gain proportional to ω

So it amplifies high-frequency sensor noise without limit

A noisy measurement becomes a violently chattering actuator

It is never implemented in its ideal form

Which is why the practical form is filtered

$\frac{K_d N s}{s+N}$. It behaves like a differentiator up to $N rad/s$ and flattens out above it, so the high-frequency gain is capped at Kd N.

P, PI, PD and PID compared

OPTIONAL

Typical trends; verify the actual plant and tuned loop

Structure	Step error	Damping	Noise	Use when
P	finite (type 0)	falls as K_p rises	low	offset is acceptable
PI	zero if stable	worse than P	low	accuracy matters, plant is slow
PD	finite (type 0)	can improve	high	speed matters; offset acceptable
PID	zero if stable	can improve	high	accuracy and damping; clean sensor

Choose the simplest controller that meets the requirements

Zero offset requires stable integral action and feasible actuator demand. Derivative action needs filtering. Check poles, zeros and margins before claiming better damping.

Concept check

EXAMPLE

Choose a next test and justify it.

"With proportional control the loop is fast, but it settles about 4 % below the setpoint and stays there."
"We added integral action. The offset is gone, but now it oscillates slowly and takes far longer to settle."

A
Remove the integral term again

B
Increase K_p to speed it back up

C
Keep integral, change something else

Predict before changing a gain. Check the cause of the oscillation.

Controller diagnosis

EXAMPLE

Preserve the accuracy requirement while testing the cause.

Integral action removed the offset. Removing it restores the original error. Keep it while investigating the oscillation.

Why not increase gain blindly

Oscillation may come from low damping or saturation.

The effect of gain depends on the plant. Predict its effect before changing it.

What to test

Try a lower integral gain. Check whether filtered derivative helps.

Recompute margins and verify the step response.

Check the model and actuator limits. Change one setting, then compare the prediction with the response.

PART 2

Controller design and implementation

Match coefficients, verify the actual response, then check actuator limits.

01 PI design and actual step response

02 PID matching and model limits

03 Saturation and anti-windup

PI design: match two coefficients

EXAMPLE

Derive the gains before trusting a simulation

Define plant, controller and target

Use $G(s) = 2$ /(s+1) and $C(s)$=Kp+Ki/s in unity feedback. The closed-loop characteristic polynomial is $s(s+1)+2(Kp\ s+Ki)$, which expands to $s^2+(1+2Kp)s+2Ki$. Choose the repeated-pole target $(s+2)^2=s^2+4s+4$.

Solve and verify

Match the s coefficient: 1+2Kp=4, giving Kp=1.5. Match the constant: 2Ki=4, giving Ki=2. Substitute back to recover s^2+4s+4. The integral term adds a loop pole at the origin, so a stable closed loop has zero final error to a unit step.

CHECK **Pole matching determines the denominator, not the shape of the whole response.**

PI design: check the numerator and peak

EXAMPLE

Repeated real poles do not guarantee zero overshoot

Construct the full transfer

With Kp=1.5 and Ki=2, the closed-loop numerator is $3s+4$, so $T(s)=(3s+4)/(s+2)^2$. For a unit step, partial fractions give $y(t) =$ 1 −e^(−2t)+t e^(−2t). The closed-loop zero changes the shape despite both poles being at −2.

Find the actual maximum

Differentiate: $y'(t)=(3-2t)e^{(-2t)}$. The output peaks at $t =$ 1.5 s, with $y =$ 1 +0.5e^(−3)≈1.0249: about 2.49% overshoot. It first reaches the setpoint at $t =$ 1 s, then briefly exceeds it. A pole-only claim of “no overshoot” would be wrong.

CHECK Always inspect zeros and the complete response after coefficient matching.

PI actuator test and final decision

OPTIONAL

A mathematically correct controller can fail an initial physical limit

Check startup and steady command

For the worked PI controller, a unit step initially has $e = 1$ and zero integral state, so $u(0^+)$=Kp=1.5. At steady state the plant needs input 0.5 to hold output one because $G(0) = 2$. If the actuator limit is ±1, this design saturates initially even though its final command is feasible.

Independent controller-design extension

Simulate the limited actuator with anti-windup; report saturation duration, overshoot, settling, load rejection, sensor-noise sensitivity and worst-case stability margin. Compare P, PI and optional filtered PID using the same plant and requirements. Recommend the simplest controller that meets all limits, or state why none does.

CHECK **The recommended controller must be feasible in both transient and steady operation.**

PID Coefficient Matching

EXAMPLE

The algebra gives a starting controller, not a released design

Match a third-order target

Use $G(s) = 1$ /$[s(s+2)]$ and ideal parallel $C(s)$=Kd s+Kp+Ki/s. The characteristic polynomial is s^3+(2+Kd)s^2+Kp s+Ki. Match the target $(s+2)(s^2+4s+8)=s^3+6s^2+16s+16$; hence Kd=4, Kp=16 and Ki=16.

Check what was not designed

The closed-loop transfer has numerator $4s^2+16s+16$, so its zeros and step shape also matter. Ideal differentiation strongly amplifies sensor noise and a setpoint step can cause a derivative kick. Implement filtered derivative on measurement, include actuator limits, then recheck margins and time response.

CHECK State whether gains use the parallel, ideal or series PID convention.

Limits of coefficient matching

OPTIONAL

The coefficient equations must have a feasible solution.

When matching is possible

PI has two adjustable gains; PID has three. Check that the coefficient equations are independent and admit usable gains.

When matching is restricted

A higher-order target usually has more conditions than a PID has free gains. Arbitrary pole placement may be impossible.

Alternative: shape the loop frequency response, then check margins and time response. Coefficient matching still needs these checks.

Saturation and integral windup

OPTIONAL

The failure that no linear analysis predicts

1. **A large command arrives**
   The error is big, so the controller demands a large u
2. **The actuator hits its limit**
   Output is limited; feedback remains connected
3. **The integrator keeps integrating**
   The error is still non-zero, so $K_i \int e\,(t)\,dt$ grows and grows
4. **The output finally reaches the setpoint**
   But the integrator holds a huge stored value
5. **The loop overshoots badly**
   It must now accumulate negative error just to unwind

The signature on a plot

A long delay before the response turns around, then a large overshoot that has nothing to do with ζ.
If you see it, suspect windup before you touch the gains.

Windup: separate error from clipping

EXAMPLE

The integrator accumulates tracking error, not the actuator command deficit

A correct numerical calculation

Suppose Ki=2 control units/(error unit·s), and error stays at +1 for a 2 s saturation interval. The integral contribution grows by Ki times the error area: 2×(1×2)=4 control units. If the requested command is 12 while the actuator can deliver 5, the seven-unit clipping gap is a different quantity.

Why anti-windup needs both signals

The persistent positive error can leave the integral state high even after output reaches the setpoint. Back-calculation uses the command gap (actual minus requested) to push that state down; conditional integration freezes it only while error would drive farther into saturation. Both require the physical limit in the model.

CHECK Plot requested u, delivered u, error and integral state separately.

Anti-windup

OPTIONAL

Two standard fixes, both simple

Conditional integration

Freeze if error deepens saturation

Resume when error drives back toward range

One line of code, and it removes most of the problem

Back-calculation

Feed the difference between the commanded u and the delivered u back into the integrator

It unwinds smoothly rather than freezing

The standard method in industrial controllers

And the check that must be run

Any simulation used for final tuning has to include the actuator limit. A linear simulation of a saturating plant is not evidence about the machine you are building.

Six checks before implementation

OPTIONAL

Check the complete controller, not only its denominator.

1. **Transient response**
   Overshoot and settling time against the specification
2. **Stability margins**
   Gain and phase margin, with crossover frequencies
3. **Noise and sensitivity**
   Peak of $|S|$, and what the actuator does with sensor noise
4. **Parameter variation**
   Recompute everything at the worst credible parameters
5. **Neglected dynamics**
   Resonances and delays outside the identified range
6. **Saturation and windup**
   Rate and amplitude limits, with anti-windup in place

A nominal pole match is a starting point. Verify the relevant response, uncertainty and implementation limits.

PART 3

Practice and implementation extensions

Optional self-study beyond the one-hour teaching route.

01 PI practice with worked answers

02 Classical tuning reference

03 Sampling and digital implementation

Independent PI practice

OPTIONAL

A changed plant and target; try before viewing the solution.

QUESTION 1

Design

$G = \frac{2}{s+4}$, unity feedback, $C = K_p + \frac{K_i}{s}$

Write the characteristic equation

Choose K_p and K_i to place both poles at -5

QUESTION 2

Explain

Why can integral action remove a stable step offset?

Why can derivative action amplify noise?

Which actuator behaviour needs a nonlinear model?

Question 1 has repeated poles

Which means the target polynomial is $(s+5)^2$, not $(s+5)(s+4)$.

Further practice beyond the main lecture route.

PI practice solution

OPTIONAL

Check the gains against the target polynomial.

Question 1: complete the algebra

With $G = 2\quad/(s+4)$ and C=Kp+Ki/s, multiply 1+CG=0 by $s(s+4)$. The polynomial becomes $s^2+(4+2Kp)s+2Ki$. Desired poles at −5 give $(s+5)^2=s^2+10s+25$. Match coefficients: 4+2Kp=10 gives Kp=3; 2Ki=25 gives Ki=12.5. Substitution reproduces the exact target denominator.

Question 2: make a design judgement

Integral action raises loop type and removes a stable step offset. Derivative action can improve damping, but a real derivative requires filtering to avoid excessive sensor-noise amplification. A linear simulation alone cannot verify saturation/windup or a response outside the validated plant frequency range; those require explicit models and tests.

CHECK **Real poles specify the denominator; inspect the closed-loop numerator before claiming zero overshoot.**

Ziegler–Nichols reference

OPTIONAL / EXTENSION

Initial tuning estimates require verification.

Controller	K_p	T_i	Td
P	$0.5\ K_u$	—	—
PI	$0.45\ K_u$	$\frac{T_u}{1.2}$	—
PID	$0.6\ K_u$	$\frac{T_u}{2}$	$\frac{T_u}{8}$

Use with care

The ultimate-gain test can drive a plant into sustained oscillation. Tuning rules provide initial settings; verify the plant assumptions, transient response and actuator limits.

Further reading beyond the main lecture route.

Digital implementation

Sampling, filtering and state updates

Sampling adds delay

Roughly half a sample period, on average

It costs phase margin, at $57.3 \cdot \omega_{gc} \cdot \frac{T}{2}$ degrees

Sample at least 20 times the crossover frequency

The integral becomes a sum

$I \leftarrow I + K_i \cdot e \cdot T_s$

Which is where windup physically lives in the code

Clamp it there

The derivative becomes a difference

$\frac{e_k - e_{k-1}}{T_s}$ amplifies quantisation noise

Filter it, and take the derivative of the measurement, not of the error

Why derivative on measurement, not on error

A step change in the setpoint makes $\frac{de}{dt}$ momentarily enormous, and the actuator gets a violent kick. Differentiating $-y$ instead removes that without changing the response to a disturbance.

Discrete PID

A schematic update; implementation needs further checks.

Position form, executed once per sample period T_s:

$$e = r - y;$$

```
I = I + Ki * e * Ts;        % clamp I here for anti-windup
D = -Kd * (y - y_prev) / Ts;   % derivative on measurement, and filtered
u = Kp * e + I + D;
u = max(umin, min(umax, u));   % the saturation that makes anti-windup necessary
```

Implementation steps still required

Add derivative filtering, update the previous measurement each sample, and implement conditional integration or back-calculation. Set initial states and test actuator limits before use.

Further reading beyond the main lecture route.

Velocity form

Incremental controller output

The idea

Compute the CHANGE in u each sample, not u itself

$$\Delta u = K_p \Delta e + K_i e T_s + \frac{K_d(\Delta^2 e)}{T_s}$$

Then $u \leftarrow u + \Delta u$

Implementation cautions

Accumulated output can still wind up during saturation

Bumpless transfer needs output tracking and initialization

It directly supports actuators with incremental commands

Equivalent forms need consistent discretization and initialization. Both still need saturation handling, filtering and state tracking.

Further reading beyond the main lecture route.

Tutorial 10

OPTIONAL

PI design, saturation and PID parameter checks

OPTIONAL PRACTICE

Tutorial 10

Q1 basic checks and conditional integration.

Q2 thermal P/PI and heater feasibility.

Q3 PID audit. Extensions A–B: filters and exact PI response.

READING CHECK

Home Reading 10

Explain P, I and D actions.

Distinguish saturation from windup.

State what a pole match cannot guarantee.

Further study: derivative filtering, digital control, state feedback and observers.

Reference: Ogata, System Dynamics 4th ed. §§10-3, 10-6 and 10-10; Modern Control Engineering 5th ed. §§8-1 to 8-5.


UNIVERSITY
OF WOLLONGONG
AUSTRALIA
Stand
for purpose
THANK YOU!